

Warum diese Entscheidungen in einem ML-Projekt?

Kompakte Projektvorbereitung für klassische Textklassifikation, Evaluation und Retraining

Ziel des Dokuments: Nicht nur scikit-learn-Code ausführen, sondern erklären können, warum eine Methode gewählt wird, wie ihre Qualität bewertet wird und welche Konsequenzen Fehler im Modell haben.

1. Warum verwende ich hier TF-IDF?

Weil die Eingaben Texte sind und ein klassischer Klassifikator numerische Merkmale benötigt.

TF-IDF wandelt Wörter bzw. Wortgruppen in numerische Vektoren um. Häufige, wenig unterscheidende Wörter erhalten weniger Gewicht; charakteristische Wörter erhalten relativ mehr Gewicht.

- Gut geeignet als einfache, schnelle Baseline für Textklassifikation.
- Mit `ngram_range=(1, 2)` können zusätzlich Wortpaare berücksichtigt werden.
- Das Ergebnis ist eine meist sehr große, aber dünn besetzte Feature-Matrix (sparse matrix).

Projektbezug: TF-IDF ist besonders nützlich, wenn nachvollziehbare klassische Textmodelle gegen komplexere Ansätze oder LLMs verglichen werden sollen.

2. Warum Logistic Regression statt K-Means?

Weil hier eine überwachte Klassifikation vorliegt: Für Trainingsbeispiele sind Zielklassen bereits bekannt.

- K-Means ist unüberwachtes Lernen: Es kennt keine Zielklassen und gruppiert Daten nur nach Ähnlichkeit.
- Logistic Regression ist überwachtes Lernen: Sie lernt die Beziehung zwischen Features und bekannten Labels.
- Für binäre Klassifikation kann Logistic Regression zusätzlich Wahrscheinlichkeiten über `predict_proba()` liefern.

K-Means: X -> Cluster

Logistic Regression: X + y -> bekannte Klasse vorhersagen

3. Warum brauche ich Ground Truth?

Ground Truth ist die fachlich als richtig festgelegte Antwort. Bei überwachten Modellen sind dies die Labels, gegen die Vorhersagen verglichen werden.

- Vorhersagen objektiv vergleichen
- Precision, Recall, F1 und Accuracy berechnen

- False Positives und False Negatives identifizieren
- Retraining und Modellversionen nachvollziehbar vergleichen

Wichtig: Wenn die Ground Truth fehlerhaft oder uneindeutig ist, kann auch die Modellbewertung irreführend sein. Labelqualität ist daher ein Teil der Modellqualität.

4. Was sagt mir ein Recall von 0,70?

Recall beantwortet die Frage: Von allen tatsächlich positiven Fällen - wie viele wurden gefunden?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Ein Recall von 0,70 bedeutet: 70 % der tatsächlich positiven Fälle wurden erkannt; 30 % wurden als False Negatives übersehen.

Projektbezug: Bei sicherheits- oder risikorelevanten Klassifikationsaufgaben kann Recall besonders wichtig sein, weil übersehene positive Fälle fachlich teuer sein können. Welche Metrik priorisiert wird, ist aber eine fachliche Entscheidung.

5. Was passiert, wenn ich den Threshold absenke?

Der Threshold ist die Entscheidungsgrenze, ab der eine positive Klasse vergeben wird.

Confidence = 0,42

Threshold 0,50 -> nicht_relevant

Threshold 0,30 -> relevant

- Niedrigerer Threshold: mehr positive Vorhersagen, häufig höherer Recall, aber potenziell mehr False Positives.
- Höherer Threshold: weniger positive Vorhersagen, häufig höhere Precision, aber potenziell mehr False Negatives.
- Das Modell wird dabei nicht neu trainiert; nur die Entscheidungsregel ändert sich.

6. Warum kann ein F1 von 1,0 bei einem einzelnen Split irreführend sein?

Ein einzelner Train/Test-Split ist nur eine mögliche Aufteilung des Datensatzes. Besonders bei kleinen Datensätzen kann ein zufällig günstiger Testsplit ein perfektes Ergebnis erzeugen, obwohl das Modell auf anderen Daten deutlich schlechter abschneidet.

- Kleine Testmengen reagieren stark auf einzelne Beispiele.
- Ähnliche Formulierungen können zufällig in Training und Test besonders günstig verteilt sein.
- Ein einzelner Wert sagt wenig über Stabilität aus.

7. Warum Cross-Validation?

Cross-Validation bewertet dasselbe Modell über mehrere systematische Datenaufteilungen. Bei 5-Fold-Cross-Validation dient jeder Fold einmal als Testmenge, während die übrigen vier Folds zum Training verwendet werden.

- Weniger abhängig von einem einzelnen zufälligen Split
- Bessere Schätzung der Generalisierungsleistung
- Schwankungen zwischen den Folds werden sichtbar
- Mittelwert und Standardabweichung liefern gemeinsam eine Aussage über Leistung und Stabilität

Beispiel: $F1 = 0,67 \pm 0,20$

-> mittlere Leistung 0,67, aber deutliche Schwankungen zwischen den Folds

8. Was mache ich mit False Negatives?

False Negatives sind tatsächlich positive Fälle, die vom Modell negativ klassifiziert wurden. Error Analysis untersucht diese Fälle einzeln und sucht nach systematischen Ursachen.

- Konkrete FN-Texte lesen und Muster sammeln
- Prüfen, ob ähnliche Formulierungen in den Trainingsdaten fehlen
- Ground-Truth-Label auf Plausibilität prüfen
- Threshold überprüfen
- Feature-Repräsentation prüfen, z. B. n-Gramme oder Embeddings
- Neue, fachlich bestätigte Beispiele gezielt in den Trainingsbestand aufnehmen

9. Wann retrainiere ich ein Modell?

Retraining ist sinnvoll, wenn neue verlässliche Trainingsdaten vorliegen oder sich Daten, Sprache, Kategorien oder fachliche Anforderungen verändern.

- Neue annotierte Daten stehen zur Verfügung
- Labels wurden korrigiert
- Wiederkehrende Fehlermuster wurden gefunden
- Datenverteilung oder Sprache hat sich verändert (Data Drift / Concept Drift)
- Neue Kategorien oder Formulierungen treten auf
- Eine neue Modellversion soll gegen die bisherige Version getestet werden

Wichtig: Retraining ist nicht automatisch eine Verbesserung. Modell V2 sollte auf einer vergleichbaren, unabhängigen Evaluation gegen Modell V1 geprüft werden.

10. Was ist Class Imbalance und warum ist das wichtig?

Class Imbalance bedeutet, dass die Klassen stark unterschiedlich häufig vorkommen, z. B. 95 % nicht_relevant und 5 % relevant.

95 % nicht_relevant

5 % relevant

Ein triviales Modell, das immer nicht_relevant vorhersagt, hätte dann 95 % Accuracy - wäre aber für die positive Klasse völlig unbrauchbar.

- Accuracy allein kann bei unausgeglichenen Klassen irreführend sein.
- Recall, Precision und F1 der seltenen Klasse werden wichtiger.
- Stratified Splits helfen, die Klassenverteilung in Train/Test ähnlich zu halten.
- Bei geeigneten Modellen kann class_weight="balanced" getestet werden.

11. Wie vergleiche ich Modelle sauber?

Modelle sollten auf denselben Daten, mit derselben Vorverarbeitung und derselben Bewertungslogik verglichen werden.

TF-IDF -> Logistic Regression

TF-IDF -> Linear SVC

TF-IDF -> Naive Bayes

- Gleiche Cross-Validation-Folds verwenden
- Gleiche Zielmetrik verwenden
- Nicht nur den besten Einzelwert betrachten, sondern Mittelwert und Streuung
- Laufzeit, Erklärbarkeit und Betriebsaufwand ebenfalls berücksichtigen

Baseline-Prinzip: Zuerst ein einfaches Modell etablieren. Ein komplexeres Modell lohnt sich nur, wenn es einen relevanten Mehrwert liefert.

12. Was ist Data Leakage?

Data Leakage entsteht, wenn Informationen in das Training gelangen, die zum Vorhersagezeitpunkt eigentlich nicht verfügbar wären oder die Zielvariable indirekt verraten.

- Beispiel: Für die Vorhersage einer SLA-Verletzung wird die tatsächliche spätere Bearbeitungsdauer als Feature verwendet.
- Das Modell wirkt dann im Test sehr gut, lernt aber eine Information, die in der realen Anwendung noch gar nicht bekannt wäre.
- Preprocessing sollte innerhalb einer Pipeline auf Trainingsdaten gelernt werden, damit keine Testinformationen einfließen.

Prüffrage: Ist jedes Feature zum echten Vorhersagezeitpunkt verfügbar? Wenn nein, besteht Leakage-Risiko.

13. Train, Validation und Test - wofür brauche ich welche Daten?

Für kleine Lernbeispiele reicht häufig Train/Test plus Cross-Validation. In größeren Projekten werden die Rollen stärker getrennt.

Datenteil	Zweck
Training	Modellparameter lernen
Validation / Cross-Validation	Modellvarianten und Hyperparameter vergleichen
Test	Abschließende, möglichst unverfälschte Bewertung

Wichtig: Das Testset sollte nicht wiederholt zur Auswahl von Modell oder Hyperparametern benutzt werden, sonst optimiert man indirekt auf den Test.

14. Was ist Hyperparameter-Tuning?

Hyperparameter werden nicht direkt aus den Daten gelernt, sondern vor dem Training festgelegt, z. B. C bei Logistic Regression oder ngram_range bei TF-IDF.

Beispiele:

`C = [0.1, 1, 10]`

`ngram_range = [(1,1), (1,2)]`

Werkzeuge wie GridSearchCV testen Kombinationen innerhalb einer Cross-Validation. Entscheidend ist, dass die Auswahl nicht anhand des finalen Testsets erfolgt.

15. Ist predict_proba() automatisch eine echte Wahrscheinlichkeit?

predict_proba() liefert modellbasierte Wahrscheinlichkeitswerte. Diese sind nicht automatisch perfekt kalibriert. Eine Ausgabe von 0,80 bedeutet nicht zwangsläufig, dass in der Realität exakt 80 % solcher Fälle positiv sind.

- Für Threshold-Entscheidungen sind Wahrscheinlichkeiten sehr nützlich.
- Bei produktiven Systemen kann zusätzlich geprüft werden, ob die Wahrscheinlichkeiten kalibriert sind.
- Confidence sollte nicht mit absoluter Sicherheit gleichgesetzt werden.

16. Wann reichen TF-IDF-Features, wann sind Embeddings interessant?

TF-IDF bewertet hauptsächlich Wörter und Wortgruppen. Embeddings repräsentieren Texte als dichte Vektoren, die semantische Ähnlichkeit besser erfassen können.

Ansatz	Stärken	Typischer Fokus
TF-IDF	Einfach, schnell, transparent, starke Baseline	Wortlaut und n-Gramme
Embeddings	Semantische Ähnlichkeit, Clustering, moderne Features	Bedeutungsähnlichkeit auch bei anderem Wortlaut

17. Warum sind Reproduzierbarkeit und Modellversionen wichtig?

In einem Projekt sollte nachvollziehbar sein, mit welchen Daten, Parametern und Code-Versionen ein Modell erzeugt wurde.

- `random_state` dokumentieren, wenn Zufall beteiligt ist
- Datensatz- und Label-Version dokumentieren
- Modellparameter und Preprocessing gemeinsam versionieren
- Metriken pro Modellversion speichern
- Nur vergleichbare Evaluationen gegenüberstellen

18. Was passiert nach dem Training in Produktion?

Eine gute Offline-Evaluation ist nur der Start. In Produktion muss beobachtet werden, ob sich Eingaben, Fehlerarten oder Klassenverteilungen verändern.

- Verteilung der Vorhersagen beobachten
- False Positives und False Negatives aus fachlicher Rückmeldung sammeln
- Drift erkennen
- Threshold oder Modell nur kontrolliert ändern
- Neue Ground Truth für spätere Retrainings aufbauen

19. Generisches Grundschema eines ML-Projekts

Daten erfassen

-> Daten prüfen / bereinigen

-> X und y definieren

-> Train / Validation / Test festlegen

-> Preprocessing

-> Baseline-Modell trainieren

-> `predict` / `predict_proba`

-> Precision / Recall / F1 / Confusion Matrix

-> Error Analysis

-> Modellvergleich / Tuning

-> finaler Test

-> Deployment

-> Monitoring

-> neue Ground Truth

-> Retraining

20. Fragen, die ich im Projekt erklären können sollte

Die folgenden Antworten sind bewusst kurz und praxisnah formuliert. Ziel ist, dass du die Punkte im Projektgespräch in eigenen Worten erklären kannst.

- **Warum ist dieses Problem Klassifikation und nicht Clustering?**

Antwort: Weil die Zielklassen bereits bekannt sind. Bei einer Klassifikation lernt das Modell aus Beispielen mit Labels wie 'relevant' und 'nicht_relevant'. Clustering wäre passend, wenn es keine bekannten Labels gäbe und das Modell nur ähnliche Fälle gruppieren sollte.

- **Woran erkenne ich, ob ich supervised oder unsupervised learning brauche?**

Antwort: Supervised Learning brauchst du, wenn zu den Trainingsdaten richtige Zielwerte oder Labels vorhanden sind. Unsupervised Learning nutzt du, wenn es keine Zielvariable gibt und du zunächst Strukturen, Gruppen oder Ähnlichkeiten in den Daten finden möchtest.

- **Welche Ground Truth steht zur Verfügung und wie zuverlässig sind die Labels?**

Antwort: Die Ground Truth sind die fachlich als richtig angesehenen Labels. Ihre Qualität hängt davon ab, wie sie entstanden sind: klare Regeln und fachlich geprüfte Annotationen sind zuverlässiger als automatisch erzeugte oder uneinheitlich vergebene Labels. Schlechte Ground Truth bedeutet: Das Modell lernt möglicherweise falsche Muster.

- **Wer erzeugt bzw. bestätigt die Ground Truth fachlich?**

Antwort: Idealerweise Fachpersonen, die die Bedeutung der Daten und die Entscheidungsregeln kennen. Data Scientists können den Prozess technisch unterstützen, sollten fachliche Wahrheit aber nicht allein festlegen, wenn dafür Domänenwissen nötig ist.

- **Welche Fehlerart ist fachlich kritischer: False Positive oder False Negative?**

Antwort: Das hängt vom Anwendungsfall ab. Ein False Positive ist ein Fehlarmed; ein False Negative ist ein übersehener positiver Fall. Wenn das Übersehen eines relevanten Falls besonders kritisch ist, sind False Negatives meist wichtiger und Recall bekommt mehr Gewicht.

- **Welche Metrik passt deshalb zur Aufgabe?**

Antwort: Wenn möglichst wenige positive Fälle übersehen werden sollen, ist Recall wichtig. Wenn positive Meldungen möglichst zuverlässig sein sollen, ist Precision wichtig. Wenn beides ausgewogen sein soll, ist F1 oft sinnvoll. Accuracy allein reicht bei unausgeglichenen Klassen häufig nicht.

- **Warum kann Accuracy bei unausgeglichenen Klassen irreführend sein?**

Antwort: Beispiel: 95 von 100 Fällen sind nicht relevant. Ein Modell, das immer 'nicht relevant' sagt, hätte 95 % Accuracy, erkennt aber keinen einzigen relevanten Fall. Deshalb betrachtet man bei solchen Daten zusätzlich Precision, Recall, F1 und die Confusion Matrix.

- **Wann ist Recall wichtiger als Precision - und wann umgekehrt?**

Antwort: Recall ist wichtiger, wenn du möglichst keinen positiven Fall verpassen willst. Precision ist wichtiger, wenn Fehlarmede teuer oder störend sind. Beispiel: Sicherheitsrelevante Hinweise finden -> eher Recall; automatisch sehr teure Folgeaktion auslösen -> eher Precision.

- **Was sagt mir die Confusion Matrix zusätzlich zu F1?**

Antwort: F1 fasst Precision und Recall in einer Zahl zusammen. Die Confusion Matrix zeigt dagegen konkret, wie viele TP, TN, FP und FN entstanden sind. Dadurch siehst du direkt, welche Art von Fehler das Modell macht.

- **Wie wurde verhindert, dass Data Leakage entsteht?**

Antwort: Alle Informationen, die erst nach dem eigentlichen Vorhersagezeitpunkt bekannt wären, dürfen nicht als Feature verwendet werden. Außerdem wird Preprocessing nur auf den Trainingsdaten gelernt. Eine Pipeline hilft dabei, weil Transformation und Modell gemeinsam korrekt auf Train- und Testdaten angewendet werden.

- **Sind alle verwendeten Features zum echten Vorhersagezeitpunkt verfügbar?**

Antwort: Das muss für jedes Feature geprüft werden. Wenn ein Merkmal erst nach dem Ereignis entsteht, darf es nicht zur Vorhersage dieses Ereignisses genutzt werden. Sonst würde das Modell Informationen aus der Zukunft verwenden und die Testwerte wären künstlich gut.

- **Warum wird Preprocessing in einer Pipeline mit dem Modell verbunden?**

Antwort: Damit dieselben Verarbeitungsschritte immer in derselben Reihenfolge auf Training und neue Daten angewendet werden. Außerdem verhindert die Pipeline leichter Data Leakage, z. B. wenn TF-IDF oder Skalierung nur auf den Trainingsdaten gelernt werden sollen.

- **Warum verwende ich stratify=y beim Train/Test-Split?**

Antwort: Damit das Verhältnis der Klassen in Trainings- und Testdaten ungefähr gleich bleibt. Wenn z. B. 20 % der Gesamtdaten 'relevant' sind, sollen auch Train- und Testset ungefähr 20 % relevante Fälle enthalten.

- **Was bewirkt random_state und warum dokumentiert man ihn?**

Antwort: random_state legt den Startwert des Zufallsgenerators fest. Mit demselben Wert bekommst du dieselbe zufällige Aufteilung erneut. Das macht Experimente reproduzierbar und Ergebnisse nachvollziehbar.

- **Wie stabil ist das Ergebnis über Cross-Validation-Folds?**

Antwort: Man vergleicht die Scores aller Folds. Liegen sie eng beieinander, ist das Modell relativ stabil. Schwanken sie stark, hängt das Ergebnis stark davon ab, welche Daten im Training bzw. Test landen.

- **Was bedeutet eine hohe Standardabweichung der Cross-Validation-Scores?**

Antwort: Sie bedeutet, dass die Modellleistung zwischen den Folds stark schwankt. Beispiel: F1-Werte von 0,95, 0,60, 0,90, 0,55 und 0,85 wären deutlich instabiler als fünf Werte um 0,82. Häufige Ursachen sind kleine Datenmengen, schwierige Fälle oder ungleich verteilte Muster.

- **Ist die Datenmenge ausreichend und sind die Klassen unausgeglichen?**

Antwort: Das prüfst du über die Anzahl der Beispiele pro Klasse und über die Stabilität der Evaluation. Sehr wenige Beispiele oder stark unterschiedliche Klassengrößen können zu instabilen Modellen führen. Bei Class Imbalance sind Precision, Recall, F1 und ggf. class_weight='balanced' besonders relevant.

- **Wie erkenne ich Overfitting?**

Antwort: Typisch ist: sehr gute Leistung auf Trainingsdaten, aber deutlich schlechtere Leistung auf unbekannten Testdaten oder Cross-Validation-Folds. Das Modell hat dann Trainingsbeispiele zu stark gelernt, statt allgemein gültige Muster zu erfassen.

- **Warum wurde dieses Modell gegenüber einer einfacheren Baseline gewählt?**

Antwort: Ein komplexeres Modell sollte einen messbaren Vorteil liefern, z. B. besseren Recall oder F1. Wenn Logistic Regression fast genauso gut ist wie ein deutlich komplexeres Modell, kann die einfache Baseline wegen Geschwindigkeit, Wartbarkeit und Erklärbarkeit die bessere Wahl sein.

- **Wie vergleiche ich Logistic Regression, Linear SVC und Naive Bayes fair?**

Antwort: Alle Modelle bekommen dieselben Trainingsdaten, dieselben Testdaten bzw. dieselben Cross-Validation-Folds und dieselbe Bewertungsmetrik. Auch das Preprocessing sollte vergleichbar sein. Nur so stammt der Unterschied im Ergebnis wirklich vom Modell und nicht von unterschiedlichen Testbedingungen.

- **Wie wurde der Threshold bestimmt?**

Antwort: Nicht einfach nach Gefühl. Man testet verschiedene Thresholds auf Validierungsdaten und schaut, wie sich Precision, Recall und F1 verändern. Danach wählt man den Wert, der zum fachlichen Ziel passt, z. B. möglichst hoher Recall bei noch akzeptabler Precision.

- **Was passiert mit Precision und Recall, wenn ich den Threshold ändere?**

Antwort: Ein niedrigerer Threshold führt meist zu mehr positiven Vorhersagen: Recall steigt oft, aber es können mehr False Positives entstehen und Precision sinken. Ein höherer Threshold macht das Modell strenger: Precision kann steigen, während Recall oft sinkt.

- **Sind die predict_proba-Werte ausreichend kalibriert oder nur als Confidence zu verstehen?**

Antwort: predict_proba liefert Modellwahrscheinlichkeiten, aber 0,80 bedeutet nicht automatisch, dass genau 80 % solcher Fälle tatsächlich positiv sind. Das wäre nur bei guter Kalibrierung zu erwarten. Ohne Kalibrierungsprüfung sollte man die Werte eher als Modell-Confidence interpretieren.

- **Wann reichen TF-IDF-Features und wann wären Embeddings sinnvoll?**

Antwort: TF-IDF reicht oft, wenn konkrete Wörter und Wortkombinationen die Klassen gut unterscheiden und eine schnelle, transparente Baseline gewünscht ist. Embeddings sind sinnvoller, wenn semantisch ähnliche Aussagen trotz unterschiedlicher Formulierungen erkannt werden sollen.

- **Wann wäre ein klassisches ML-Modell sinnvoller als ein LLM?**

Antwort: Wenn die Aufgabe klar definiert ist, ausreichend gelabelte Daten vorhanden sind und Geschwindigkeit, Kosten, Reproduzierbarkeit oder Erklärbarkeit wichtig sind. Für eine feste Textklassifikation kann TF-IDF + Logistic Regression oder Linear SVC oft einfacher und günstiger sein als ein LLM.

- **Welche Fehlerfälle treten wiederholt auf und was lerne ich daraus?**

Antwort: Bei der Error Analysis gruppierst du ähnliche FP- und FN-Fälle. Wiederholen sich bestimmte Formulierungen oder Themen, zeigt das mögliche Lücken in Trainingsdaten, Labels, Features oder Thresholds. Daraus leitest du gezielte Verbesserungen ab.

- **Wie werden False Positives und False Negatives systematisch untersucht?**

Antwort: Die Fehler werden mit Text, Ground Truth, Modellvorhersage und Confidence gespeichert und getrennt nach FP und FN analysiert. Danach suchst du wiederkehrende Muster: unbekannte Formulierungen, falsche Labels, zu wenig Trainingsbeispiele oder einen ungeeigneten Threshold.

- **Wie werden neue Ground-Truth-Daten gesammelt und für Retraining verwendet?**

Antwort: Neue reale Fälle werden fachlich annotiert und versioniert. Nach Qualitätsprüfung werden sie zum Trainingsdatensatz hinzugefügt. Anschließend wird ein neues Modell trainiert und gegen das bisherige Modell auf einem festen Testset verglichen.

- **Wann ist Retraining wirklich notwendig und wann reicht eine Threshold-Anpassung?**

Antwort: Threshold-Anpassung reicht, wenn die Wahrscheinlichkeiten grundsätzlich sinnvoll sind, aber die gewünschte Balance zwischen Precision und Recall geändert werden soll. Retraining ist nötig, wenn neue Muster, neue Klassen, Data Drift oder systematische Modellfehler auftreten, die das vorhandene Modell gar nicht gelernt hat.

- **Wie wird Modell V2 fair gegen Modell V1 verglichen?**

Antwort: Beide Modelle müssen auf demselben unveränderten Testset und mit denselben Metriken bewertet werden. Idealerweise werden zusätzlich Fehlerfälle und Laufzeit verglichen. Nur so lässt sich sagen, ob V2 tatsächlich besser ist.

- **Welche Daten-, Code- und Modellversion gehört zu einem bestimmten Ergebnis?**

Antwort: Für jedes Experiment sollten Datensatzversion, Codeversion, Modellparameter und erzeugtes Modell dokumentiert werden. So kann später nachvollzogen werden, womit ein bestimmter F1-Wert oder eine bestimmte Vorhersage entstanden ist.

- **Wie wird nach Deployment überwacht, ob das Modell schlechter wird?**

Antwort: Man protokolliert Vorhersagen, Confidence-Werte und - sobald verfügbar - echte Ground-Truth-Ergebnisse. Daraus werden regelmäßige Metriken und Fehlerquoten berechnet. Verschlechtern sie sich über die Zeit, muss die Ursache untersucht werden.

- **Wie erkenne ich Data Drift oder veränderte Formulierungen in neuen Daten?**

Antwort: Man vergleicht neue Daten mit den Trainingsdaten, z. B. Klassenverteilung, Wort- oder Feature-Verteilungen, Embeddings und Confidence-Verteilungen. Tauchen neue Begriffe, Themen oder deutlich andere Verteilungen auf, kann das ein Hinweis auf Drift sein.

- **Welche Metriken und Fehlertypen sollten im Betrieb regelmäßig überwacht werden?**

Antwort: Mindestens Anzahl der Vorhersagen pro Klasse, Precision, Recall, F1, False Positives, False Negatives und Confidence-Verteilung - soweit Ground Truth verfügbar ist. Zusätzlich können Datenmenge, Laufzeit, Fehlerraten der API und Drift-Indikatoren überwacht werden.

21. Wichtige Begriffe und Metriken – sehr einfach erklärt

Für die folgenden Beispiele nehmen wir 100 Testfälle. 40 davon sind tatsächlich relevant und 60 nicht relevant. Das Modell liefert: 30 True Positives, 10 False Negatives, 5 False Positives und 55 True Negatives.

21.1 Die vier Grundbegriffe: TP, TN, FP und FN

True Positive (TP): Tatsächlich positiv/relevant UND vom Modell positiv/relevant erkannt. Beispiel: Ein wirklich relevanter Text wird als relevant erkannt.

True Negative (TN): Tatsächlich negativ/nicht relevant UND vom Modell negativ/nicht relevant erkannt. Beispiel: Ein normaler Text wird korrekt als nicht relevant erkannt.

False Positive (FP): Tatsächlich negativ, aber das Modell meldet positiv. Einfach gesagt: Fehllarm. Beispiel: Ein harmloser Text wird als relevant markiert.

False Negative (FN): Tatsächlich positiv, aber das Modell meldet negativ. Einfach gesagt: Das Modell übersieht einen wichtigen Fall. Beispiel: Ein relevanter Text wird als nicht relevant eingestuft.

Anschauliche Confusion Matrix für unser Beispiel:

	Vorhersage	
	nicht relevant	relevant
Tatsächlich nicht relevant	55	5
Tatsächlich relevant	10	30

21.2 Accuracy – Wie viel ist insgesamt richtig?

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Beispiel: $(30 + 55) / 100 = 0,85 = 85 \%$. Das Modell liegt also bei 85 von 100 Fällen richtig.

Wichtig: Accuracy kann bei unausgeglichene Klassen täuschen. Wenn 95 von 100 Fällen nicht relevant sind, könnte ein Modell immer „nicht relevant“ sagen und hätte 95 % Accuracy – obwohl es keinen einzigen relevanten Fall findet.

21.3 Precision – Wenn das Modell „relevant“ sagt: Wie oft stimmt das?

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Beispiel: $30 / (30 + 5) = 0,857 \approx 86 \%$. Von 35 Alarmen des Modells waren 30 wirklich relevant.

Hohe Precision bedeutet: wenige Fehllarme / wenige False Positives.

21.4 Recall – Wie viele der wirklich relevanten Fälle finde ich?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Beispiel: $30 / (30 + 10) = 0,75 = 75 \%$. Von 40 tatsächlich relevanten Fällen findet das Modell 30 und übersieht 10.

Hoher Recall bedeutet: wenige relevante Fälle werden übersehen / wenige False Negatives.

Andere Bezeichnung: Sensitivity oder Trefferquote der positiven Klasse meint im Kern dasselbe wie Recall.

21.5 F1-Score – Kompromiss zwischen Precision und Recall

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Beispiel: Precision $\approx$ 0,86 und Recall = 0,75 $\rightarrow$ F1 $\approx$ 0,80.

Der F1-Score wird hoch, wenn Precision UND Recall gut sind. Ein Modell kann deshalb nicht einfach einen sehr hohen Wert bei einer Kennzahl nutzen, um einen sehr schlechten Wert bei der anderen vollständig zu verstecken.

21.6 Specificity – Wie gut erkenne ich die negativen Fälle?

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

Beispiel: 55 / (55 + 5) = 0,917 $\approx$ 92 %. Von 60 tatsächlich nicht relevanten Fällen erkennt das Modell 55 korrekt.

Während Recall auf die positive Klasse schaut, betrachtet Specificity die negative Klasse.

21.7 False Positive Rate (FPR) – Wie oft erzeuge ich Fehlalarme?

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN}) = 1 - \text{Specificity}$$

Beispiel: 5 / (5 + 55) = 0,083 $\approx$ 8 %. Rund 8 % der tatsächlich negativen Fälle werden fälschlich als positiv markiert.

21.8 False Negative Rate (FNR) – Wie oft übersehe ich positive Fälle?

$$\text{FNR} = \text{FN} / (\text{FN} + \text{TP}) = 1 - \text{Recall}$$

Beispiel: 10 / (10 + 30) = 0,25 = 25 %. Ein Viertel der tatsächlich relevanten Fälle wird übersehen.

21.9 Balanced Accuracy – sinnvoll bei unausgeglichenen Klassen

$$\text{Balanced Accuracy} = (\text{Recall} + \text{Specificity}) / 2$$

Beispiel: (0,75 + 0,917) / 2 $\approx$ 0,833 = 83 %. Beide Klassen werden dadurch gleich stark berücksichtigt.

Das ist oft aussagekräftiger als normale Accuracy, wenn eine Klasse viel häufiger vorkommt als die andere.

21.10 Support – Wie viele echte Beispiele einer Klasse gibt es?

Support ist keine Qualitätsmetrik. Er sagt nur, wie viele tatsächliche Beispiele einer Klasse im ausgewerteten Datensatz vorkommen. In unserem Beispiel: Support relevant = 40, Support nicht relevant = 60.

Warum wichtig? Ein F1 von 1,0 bei nur 2 Testfällen ist viel weniger belastbar als ein F1 von 0,95 bei 2.000 Testfällen.

21.11 Macro, Weighted und Micro Average im classification_report

Macro Average: Metrik für jede Klasse berechnen und anschließend einfach mitteln. Jede Klasse zählt gleich stark – auch eine seltene Klasse.

Weighted Average: Wie Macro, aber große Klassen bekommen entsprechend ihrem Support mehr Gewicht.

Micro Average: Erst TP/FP/FN über alle Klassen zusammenzählen und daraus die Metrik berechnen. Dadurch dominieren häufige Klassen stärker.

Für unausgeglichene Multi-Class-Probleme ist Macro-F1 oft interessant, weil eine seltene Klasse nicht von einer großen Klasse überdeckt wird.

21.12 Threshold – ab wann sage ich „positiv“?

Der Threshold ist die Entscheidungsgrenze. Beispiel: Das Modell gibt für „relevant“ 0,72 aus. Bei Threshold 0,50 → relevant. Bei Threshold 0,80 → nicht relevant.

Threshold senken: meist mehr positive Vorhersagen → Recall kann steigen, aber mehr False Positives sind möglich. Threshold erhöhen: meist weniger positive Vorhersagen → Precision kann steigen, aber mehr False Negatives sind möglich.

21.13 Confidence / predict_proba – wie sicher wirkt die Modellentscheidung?

predict_proba() liefert modellbasierte Wahrscheinlichkeitswerte. Beispiel: relevant = 0,72, nicht_relevant = 0,28. Für die Threshold-Entscheidung kann 0,72 als Confidence für relevant verwendet werden.

Wichtig: 0,72 bedeutet nicht automatisch, dass das Modell in der Realität exakt in 72 % solcher Fälle richtig liegt. Dafür müsste die Wahrscheinlichkeit gut kalibriert sein.

21.14 ROC-AUC – Wie gut trennt das Modell positive und negative Fälle?

ROC-AUC bewertet die Trennfähigkeit über viele mögliche Thresholds. Grob: 0,5 entspricht ungefähr Zufall, 1,0 wäre perfekte Trennung. Ein Wert von 0,90 bedeutet eine starke Trennfähigkeit, sagt aber noch nicht, welcher konkrete Threshold fachlich sinnvoll ist.

Bei stark unausgebalancierten Klassen kann zusätzlich die Precision-Recall-Kurve bzw. PR-AUC aussagekräftiger sein, weil sie stärker auf die positive Klasse fokussiert.

21.15 PR-AUC – Precision und Recall über viele Thresholds

PR-AUC fasst zusammen, wie sich Precision und Recall über verschiedene Thresholds verhalten. Sie ist besonders hilfreich, wenn positive Fälle selten sind und genau diese seltene Klasse wichtig ist.

21.16 Overfitting und Underfitting – zwei typische Modellprobleme

Overfitting: Das Modell lernt die Trainingsdaten zu genau. Training sehr gut, unbekannte Testdaten deutlich schlechter. Bildlich: Das Modell lernt Antworten auswendig statt die Regel zu verstehen.

Underfitting: Das Modell ist zu einfach oder hat zu wenig gelernt. Training UND Test sind schlecht.
Bildlich: Das Modell hat die Aufgabe noch nicht ausreichend verstanden.

21.17 Class Imbalance – wenn eine Klasse viel häufiger ist

Beispiel: 990 nicht relevante und nur 10 relevante Fälle. Ein Modell, das immer „nicht relevant“ sagt, hätte 99 % Accuracy, aber Recall für relevant = 0 %. Deshalb immer Klassenverteilung, Confusion Matrix, Precision, Recall und F1 mit betrachten.

21.18 Die wichtigsten Metriken als Kurz-Merksatz

- Accuracy: Wie viel ist insgesamt richtig?
- Precision: Wenn das Modell positiv sagt – wie oft stimmt es?
- Recall: Von allen echten positiven Fällen – wie viele finde ich?
- F1: Sind Precision und Recall gemeinsam gut?
- Specificity: Von allen echten negativen Fällen – wie viele erkenne ich korrekt?
- False Positive: Fehllalarm.
- False Negative: Wichtiger Fall wurde übersehen.
- Support: Wie viele echte Beispiele wurden überhaupt bewertet?
- Cross-Validation: Ist die Leistung über verschiedene Teilmengen stabil?

21.19 Komplettes Mini-Beispiel auf einen Blick

100 Fälle: 40 relevant, 60 nicht relevant. Modell: TP=30, FN=10, FP=5, TN=55.

- Accuracy = $85 / 100 = 85 \%$
- Precision = $30 / 35 \approx 86 \%$
- Recall = $30 / 40 = 75 \%$
- F1 $\approx 80 \%$
- Specificity = $55 / 60 \approx 92 \%$
- FPR = $5 / 60 \approx 8 \%$
- FNR = $10 / 40 = 25 \%$

Interpretation: Das Modell erzeugt relativ wenige Fehllalarme, übersieht aber noch 25 % der relevanten Fälle. Wenn das Übersehen fachlich besonders kritisch ist, müsste man z. B. Threshold, Daten, Features oder Modell gezielt auf höheren Recall prüfen.

22. Merksatz

Ein gutes ML-System ist nicht einfach das Modell mit dem höchsten Score. Entscheidend sind verlässliche Daten und Labels, passende Metriken, stabile Validierung, nachvollziehbare Fehleranalyse, saubere Modellvergleiche und kontrolliertes Retraining.